Milestone-B API Test Script Development and Baseline Execution

Melbourne Urban Forest API checks

1 Document details

Field	Value
Prepared by	Darvin Anorma Mangampo
Assessment period	14–15 October 2026 (UTC+08:00)
Run date	15 October 2026 (UTC+08:00)
Service	City of Melbourne Urban Forest data, Explore API v2.1
Run command	node ../../../node_modules/@playwright/test/cli.js test --config playwright.config.ts --reporter=list,html
Source and evidence commit	92c23d4c9fe7b81abb8a05eef51a6dcbc88805b0
Test scope	Read-only GET requests to the public Urban Forest API
Document date	15 October 2026

2 Purpose and testing context

In Milestone-A, I used Playwright to confirm API access and examine the tree fields returned. The saved log showed HTTP 200 for the genus filter and second sorted page, but it did not retain how many trees those responses contained. During the complete Milestone-B run, I checked their row counts alongside the other conditions in the eight-row matrix. I use the complete T01–T08 result documented here as the baseline for this report.

I began this API testing project this year (2026) as part of my upskilling after The Test Tribe Playwright course. The project is separate from my previous role. No client requirements were supplied, so I used the City's published API rules and the responses I observed during the 13 October preflight. For the lookup and total checks, I compared values returned within the same run.

3 Test execution approach

I kept the eight checks in one TypeScript specification. They use Playwright's request fixture for GET calls and expect for assertions. I put tree data and consistency in one test.describe group, with input limits and invalid fields in the other. Before the checks, I requested five trees sorted by com_id and chose a row with both com_id and genus populated. I used those current values for the lookup and genus filter. The suite also checked two ordered pages, input limits, invalid fields, the two tree totals and a high tree ID. T01 carries an @smoke tag. For T05, I used a short JSON file containing two page-size values: limit=100, the published maximum, and limit=101, one above it. I checked whether the API accepted the first request and rejected the second.

I used one worker and paused for 700 ms before each GET request. Retries were off, so a failing check would remain visible in the Playwright HTML report. The ten outcomes are in the saved HTML report for this run.

4 Result of the 15 October run

I ran the suite on my computer on 15 October 2026 (UTC+08:00). Playwright reports ten test executions because two of the eight conditions each have two input cases. T01–T04, T07 and T08 ran once each. T05 ran with limit=100 and limit=101. T06 ran with an unknown field in where and then in order_by. All ten executions passed, and none was skipped. During the run, I sent 13 read-only GET requests. Ten returned HTTP 200. The three deliberately invalid requests returned HTTP 400. No request returned HTTP 403 or HTTP 429, and I did not encounter an access challenge.

Checks	Observed result and meaning
T01	T01 returned HTTP 200 JSON with five tree rows and a numeric total_count. The saved sample had com_id 1023218 and genus Platanus. Its field names and values are in the test-evidence file.
T02	This test requested the first two pages in com_id order, with five trees on each page. It checked that the ten returned IDs were in ascending text order with no duplicates. The saved result contains both ordered lists and ID sets.
T03	The lookup for com_id 1013381 gave the same ID and genus Ulmus as the tree I found during setup. I checked both fields to confirm that the lookup returned the same tree.
T04	The genus filter returned five trees, all labelled Ulmus. I checked the genus values in the rows as well as the HTTP status.
T05	limit=100 returned HTTP 200 and 100 rows. Moving one above the published page-size maximum to limit=101 returned HTTP 400 JSON, with a message naming the limit. This pair covered both sides of the documented boundary.
T06	I ran this check with two invalid-field inputs, one in where and one in order_by. Both requests returned HTTP 400 JSON. I checked the status and confirmed that each error message identified the invalid field or clause.
T07	The data response reported total_count 82,064. Metadata reported records_count 82,064 in the same run. The figures agreed, though they do not verify the accuracy of the underlying tree inventory.
T08	The query for com_id 999999999 returned HTTP 200, total_count 0 and an empty results array. That was the response for this ID with no match in the 15 October run.

5 Interpretation and limits of the result

The complete run showed that the second sorted page and genus filter each contained five trees, adding row counts that the earlier access log did not retain. The lookup and filter checks compare returned values, not just response status. Both endpoints reported 82,064 trees from the same City dataset. Their matching counts show that the two API responses agreed during this run. They do not check whether individual entries are accurate. The City dataset page lists a monthly source-data update frequency, so a later total may differ.

Every assertion in T01–T08 met its matrix expectation. The result comes from one run of a live public service and does not establish long-term availability.

In Milestone-A, I checked the 10,000 paging boundary. With limit=100 and offset=9900, the API returned HTTP 200 and 100 rows. Increasing the offset to 9901 returned HTTP 400 with an error saying the sum must be no greater than 10,000. The City of Melbourne's API specification advises that the sum should be less than 10,000, while the service accepted exactly 10,000 in my check. I kept this deep-paging check outside Milestone-B's eight conditions. The suite checked tree data and two nearby five-row pages. T05 covered the separate 100/101 page-size pair. All ten executions passed, but none tested the separate deep-paging boundary where offset + limit = 10,000. This run therefore does not resolve the difference observed in Milestone-A between the published guidance and the API response at that boundary.

The HTML report shows the passes. The test-evidence file keeps T01's sample fields, both ordered ID lists from T02 and every genus value compared in T04. Melbourne defines com_id as text. T02 compared the ten returned IDs as text. They were in ascending order for this sample. The check does not establish how every possible text ID would sort.

6 Supporting evidence

The Milestone-B README links to the files below. Together, they show the test inputs, the checks I ran, their results and the limits of my interpretation. I ran the suite before making the Milestone-B commit. The run summary records hashes of the source files I used. Commit 92c23d4c9fe7b81abb8a05eef51a6dcbc88805b0 contains the test specification, boundary-case JSON, updated Playwright configuration, run summary, test evidence, console output and HTML report.

The README, this report and findings note are separate supporting documents.

Repository path	Purpose
tests/urban-forest.api.spec.ts	The eight conditions and their value checks.
test-data/boundary-cases.json	The page-size values 100 and 101 used in T05.
evidence/test-plan.md evidence/test-matrix.md	Scope and expected-result basis prepared in Milestone-A.
evidence/Milestone-B/html-report/index.html	The saved HTML report shows the results of ten Playwright executions covering the eight matrix conditions.
evidence/Milestone-B/baseline-run-summary.json evidence/Milestone-B/test-evidence.json	Summary date, command, versions, overall result and source hashes. Test evidence keeps request details and sampled values.
evidence/findings.md	What the 15 October responses showed and what one run cannot establish.

7 Sources and scope of this evidence

The City of Melbourne Urban Forest dataset page and the Explore API v2.1 reference provide the published fields and query rules. In the matrix, P means a rule from the provider and O refers to the responses I observed during the 13 October preflight. I use R for comparisons between responses in the same run. I consulted section 1.4.1 of the ISTQB Foundation Level syllabus for its description of preparing and running tests. Section 4.2.2 informed the 100/101 page-size pair.

This work covers GET requests to one public dataset. It does not include changes to the service, browser-interface testing, load testing or a check of every tree entry. The pass count describes the checks and data available on 15 October 2026.

8 Conclusion

On 15 October 2026, all eight conditions ran and ten executions passed. The second page and genus filter contained trees, filling the earlier row-count gap. I checked small API responses rather than every tree entry. The matching totals came from the same published dataset, so they do not verify the accuracy of individual entries. The live data may change.